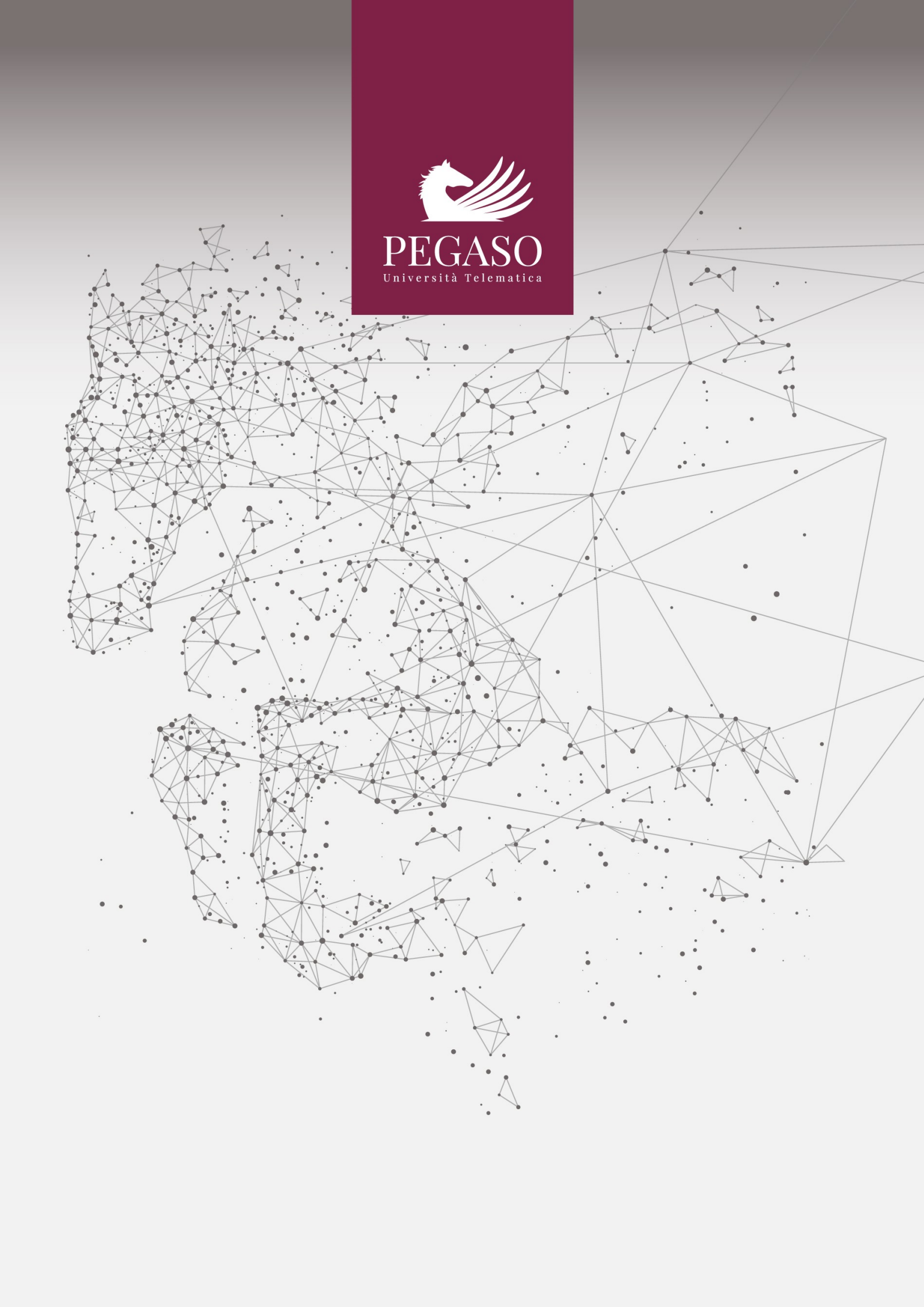

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. VISIBILITÀ 3
2. EREDITARIETÀ 10
3. EREDITARIETÀ E INCAPSULAMENTO 15
4. EREDITARIETÀ E COSTRUTTORI 20
BIBLIOGRAFIA 26

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Visibilità

Il concetto di incapsulamento è strettamente connesso al concetto di visibilità del componente software (classe, variabile, metodo) che può essere regolata in diversi modi, in particolare tramite i modificatori di accesso, il reference this, i package, il modificatore static (ci sarebbe anche final ma non è connesso alla visibilità). I package sono stati analizzati in precedenza così come i modificatori di accesso e il modificiatore static . Vedremo in seguito il reference this.

a. Reference this

Java introduce una parola chiave che per definizione coincide con il reference dell'oggetto corrente: this.

In generale, nella definizione di un metodo all'interno di una classe la variabile di istanza viene riferita direttamente. Per esempio, nella definizione del metodo getGiorno di una classe Data che astrae il concetto di data.

Si ha return giorno; dove giorno è una variabile di istanza della classe. Lo stesso vale per metodi di istanza. Ovvero si può fare riferimento a variabili e metodi d'istanza direttamente con il loro nome come mostrato in Fig. 1.

```
…
public int getGiorno () {
        return giorno;
}
…
```

Fig. 1: Uso variabile di istanza possibile attraverso il loro nome senza indicare il nome dell'istanza dell'oggetto

D'altra parte ogni variabile di istanza, come giorno, appartiene ad un oggetto.

Ma il nome dell'istanza della classe (oggetto) su cui è invocata non potrebbe essere usato visto che siamo nella parte di definizione della classe e non nel main.

Quindi in genere si sottintende l'oggetto omettendone il nome. Tuttavia, l'oggetto (a cui appartiene la variabile di istanza) lo si può esplicitamente referenziare se si vuole usando il reference this.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il nome sottinteso di questo oggetto è this (letteralmente "questo"). Per esempio, l'assegnamento precedente è equivalente a return this.giorno come mostrato in Fig. 2

```
…
public int getGiorno () {
        return this.giorno;
}
…
```

Fig. 2: Uso variabile di istanza tramite reference this

Il reference this viene implicitamente aggiunto nel bytecode compilato, per referenziare esplicitamente l'oggetto a cui si riferisce la variabile di istanza.

La parola chiave this rappresenta l'oggetto che riceve l'invocazione del metodo e quindi a cui si riferisce una variabile di istanza.

In generale non si usa this, perché si può fare riferimento a variabili e metodi d'istanza direttamente con il loro nome.

Quindi non importa specificarlo ed è aggiunto in automatico dalla JVM. Occorre usarlo in casi di ambiguità ed è per questo che ne parliamo.

Vediamo quali sono.

In Java si fa la distinzione tra variabili di istanza e variabili locali le quali vengono memorizzate in due aree di memoria differenti, rispettivamente la Heap Memory e la Stack Memory.

Questo fa sì che il compilatore consente di dichiarare nella stessa classe, una variabile locale (o un parametro di un metodo) ed una variabile d'istanza anche usando lo stesso identificatore.

Tuttavia spesso capita di fare passaggi di parametri nei metodi al fine di inizializzare variabili d'istanza. Sino ad ora, per il parametro passato siamo stati spinti ad inventare un identificatore differente da quello della variabile d'istanza da inizializzare.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Invece possiamo usare il reference this, per risolvere eventuali ambiguità. Per esempio consideriamo la seguente classe Cliente definita come mostrato in Fig. 3:

```
public class Cliente {

        private String nome, indirizzo, numeroDiTelefono;

        public void setNumeroDiTelefono (String numeroDiTelefono) {
        numeroDiTelefono = numeroDiTelefono;
        }
        public void setIndirizzo (String indirizzo) {
        indirizzo = indirizzo;
        }
        public void setNome (String nome) {
        nome = nome;
        }
//metodi get omessi…
}
```

Fig. 3: Esempio di ambiguità nell'uso delle variabili di istanza

Consideriamo per esempio il metodo setNome.

Chiaramente l'intenzione è quella di assegnare a nome, variabile di istanza, il valore dì nome parametro del metodo.

A prima vista può sembrare illegale, poiché all'interno del corpo del metodo si hanno due variabili con lo stesso identificativo: la variabile di istanza e la variabile locale al metodo. In realtà il compilatore non produce nessun messaggio di errore.

Infatti, all'interno di un blocco è possibile avere due variabili con lo stesso identificativo, a patto che una sia una variabile di istanza e l'altra una variabile locale al metodo (o anche un parametro) come si ha in questo caso.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Se però si tenta di eseguire il seguente codice:

```
…
Cliente unCliente = new Cliente();
unCliente.setNome(Carlo);
System.out.println(unCliente.getNome());
// il risultato è nome=«»
…
```

L'output prodotto sarà la stringa vuota «». In altre parole, non viene assegnato il valore Carlo alla variabile di istanza nome e il suo valore rimane «», il valore assegnato per default ai tipi di tipo String.

Tale comportamento è dovuto al fatto che nell'istruzione di assegnamento: nome=nome entrambe le variabili vengono considerate come la variabile locale al metodo non trovando riferimenti espliciti.

Praticamente il codice non fa altro che riassegnare alla variabile locale il suo valore.

Poiché l'intento era quello di assegnare alla variabile di istanza dell'oggetto invocante il metodo (setNome) il valore passato come argomento al metodo, occorre esplicitare che la prima variabile è in realtà la variabile di istanza e non la variabile locale.

Quindi this serve per distinguere tra una variabile di istanza e una variabile locale (o parametro) all'interno di un metodo.

Tramite la parola chiave this, intendiamo che la variabile referenziata appartiene all'istanza dell'oggetto che riceve l'invocazione del metodo.

Di conseguenza, per indicare che la prima variabile dell'assegnamento è la variabile di istanza si modifica il codice del metodo anteponendo alla variabile di istanza il reference this come mostrato nel codice rivisto per la classe Cliente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Di conseguenza la variabile non referenziata sarà il parametro del metodo senza che vi sia ambiguità come mostrato nella classe Cliente rivista come mostrato di seguito:

```
…
public class Cliente {

private String nome, indirizzo, numeroDiTelefono;
public void setNumeroDiTelefono (String numeroDiTelefono) {
        this.numeroDiTelefono = numeroDiTelefono;
}
public void setIndirizzo (String indirizzo) {
        this.indirizzo = indirizzo;
}
public void setNome (String nome) {
        this.nome = nome;
}
//metodi get omessi…
}
```

Con questa modifica, a fronte dell'esecuzione delle istruzioni:

```
…
Cliente unCliente = new Cliente();
unCliente.setNome(Carlo);
System.out.println(unCliente.getNome());
// il risultato è Carlo
…
```

L'output sarà Carlo, che è esattamente quello che si voleva ottenere.

In generale per riferirsi a una variabile di istanza occorre utilizzare la seguente sintassi:

this.nome_della_variabile_di_istanza dove la parola chiave this rappresenta l'oggetto che riceve l'invocazione del metodo.

Per quanto detto this rappresenta l'oggetto corrente e può quindi referenziare le variabili d'istanza.

Naturalmente con this è anche possibile invocare metodi sull'oggetto corrente. Ad esempio, sempre nella classe Cliente vediamo di seguito come nel metodo costruttore sono stati invocati i metodi set sfruttando il reference this.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
…
public class Cliente {

private String nome, indirizzo, numeroDiTelefono;

public Cliente (String nome, String indirizzo, String numeroDiTelefono) {
        this.setNome (nome);
        this.setIndirizzo(indirizzo);
        this.setNumeroDiTelefono(numeroDiTelefono)
}

//metodi pubblici set delle variabili di istanza come precedenti
//metodi get omessi
}
```

In questo caso però se non avessimo anteposto il reference this ai metodi set, avremmo comunque ottenuto lo stesso risultato visto che il compilatore lo avrebbe inserito implicitamente (e non siamo nel caso di ambiguità visto prima).

Più interessante è l'uso che si fa solitamente del reference this con i costruttori. La sintassi cambia visto che per un costruttore l'operatore dot non è utilizzabile.

Consideriamo sempre la classe Cliente modificata come segue:

```
…
public class Cliente {

private String nome, indirizzo, numeroDiTelefono;

public Cliente (String nome, String indirizzo) {
        this nome, indirizzo, «sconosciuto»);
}

public Cliente (String nome, String indirizzo, String numeroDiTelefono) {
        this.setNome (nome);
        this.setIndirizzo(indirizzo);
        this.setNumeroDiTelefono(numeroDiTelefono)
}

//metodi pubblici set delle variabili di istanza come precedenti
//metodi get omessi
}
```

con il costruttore che prende in ingresso 3 valori corrispondenti alle 3 variabili di istanza e li imposta invocando i relativi metodi set.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In aggiunta definiamo un altro costruttore che prende in ingresso 2 valori per nome ed indirizzo e li imposta invocando l'altro costruttore con tre parametri di tipo stringa dopo la parola chiave this che coincidono con i parametri del secondo costruttore.

In questo caso si è riusato il codice del secondo costruttore senza scrivere di nuovo le stesse istruzioni. Questo tipo di pratica è raccomandato.

L'utilizzo della parla chiave this per invocare un costruttore è consentito solo all'interno di un altro costruttore. L'invocazione di un altro costruttore è consentita come prima istruzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Ereditarietà

Come tutti i paradigmi che caratterizzano l'Object Oriented anche l'ereditarietà è ispirata a qualcosa che esiste nella realtà.

Nel mondo reale classifichiamo i concetti sia fisici che astratti in classi e sottoclassi.

Per esempio un cane è un animale così come lo è un moscerino, un aereo è un veicolo così come lo è un'auto, la chitarra è uno strumento musicale così come lo è un flauto.

In OO (e quindi anche Java) l'ereditarietà è la caratteristica che mette in relazione di estensibilità più classi che hanno caratteristiche comuni.

Per esempio la classe Cane e la classe Moscerino saranno sottoclassi ed estenderanno entrambe la classe Animale andando a definirne delle proprietà specifiche.

Al tempo stesso la classe Animale sarà la superclasse ed esprimerà le sole caratteristiche comuni a tutti gli animali, es. categoria (mammifero, rettile, volatile….).

Si parla di generalizzazione se a partire da un certo numero di classi si definisce una superclasse che ne raccoglie le caratteristiche comuni. Viceversa si parla di specializzazione quando, partendo da una classe si definiscono una o più sottoclassi allo scopo di ottenere oggetti più specializzati.

Si definisce una classe derivata, o sottoclasse, partendo dalla definizione di un'altra classe già definita e aggiungendo o modificando i metodi e le variabili di istanza necessari.

La classe di partenza è detta classe base o superclasse.

La classe derivata eredita tutte le variabili di istanza, le variabili statiche e tutti i metodi pubblici della classe base e può aggiungere variabili proprie e metodi propri.

Il riuso anche facilitato da ereditarietà. Il risultato immediato infatti è la possibilità di ereditare codice già scritto, e quindi di gestire insiemi di classi collettivamente, giacché accomunate da alcune caratteristiche.

Le caratteristiche della superclasse siano automaticamente disponibili per la sottoclasse.

La superclasse è riusata nelle funzionalità base (a comune), la sottoclasse definisce poi operazioni e funzionalità più specifiche.

Anche il codice a comune fra più sottoclassi può essere messo a comune nella superclasse senza riscriverlo due volte uguale o quasi nelle due sottoclassi.

Sebbene l'ereditarietà sia un argomento agevole da comprendere nei principi di base, non è semplice utilizzarla in modo corretto e al pieno delle sue potenzialità.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La parola chiave extends ci permette di implementare l'ereditarietà:

```
public class nome_sottoclasse extends nome_superclasse
{
      // variabili aggiuntive
      // metodi aggiuntivi e/o modificati
}
```

C'è un modo molto semplice per capire quando si deve implementare l'ereditarietà.

E' necessario porsi la semplice domanda «un oggetto della candidata sottoclasse è un oggetto della candidata superclasse»? In altre parole occorre capire se c'è una relazione del tipo «è un» («is a» in inglese) tra le due classi candidate. Se la risposta alla domanda è positiva l'ereditarietà si deve applicare, altrimenti no.

Questo test permette di capire una vera relazione di ereditarietà tra due classi proprio a livello concettuale così come deve essere o, in altri termini, di smascherare situazioni che potrebbero portarci a pensare che ci sia una relazione di ereditarietà quando invece non c'è.

Ad esempio la presenza di attributi comuni fra due classi potrebbe far pensare che ci sia una relazione di ereditarietà ma non sempre è così.

Prendiamo come esempio di classi Studente e Professore (mostrate di seguito) pensate per una ipotetica applicazione per l'archiviazione di informazioni di un'università.

```
public class Studente {
        public String nome;
        public String cognome;
        public String dataDiNascita;
        public int numeroEsamiSostenuti;
        //…
}
public class Professore extends Studente {
        public String materia;
        //…
}
```

La classe Studente definisce fra gli altri attributi come nome, cognome, data di nascita.

Dal momento che anche la classe Professore ha nome, cognome e data di nascita come attributi si potrebbe pensare di derivare la classe Professore dalla classe Studente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ma un professore non è uno studente (né viceversa), quindi non è rispettata la relazione «è un» e quindi non è opportuna una relazione di ereditarietà fra di essi.

Se mettessimo in relazione di ereditarietà in modo errato due classi, in un primo momento risparmieremmo del codice, in seguito tuttavia sorgerebbero problemi quando tenteremmo di usare un oggetto professore in modo analogo ad un oggetto studente proprio perché non c'è la relazione a livello concettuale.

Inoltre la classe Studente potrebbe evolvere in modo divergente dal concetto Professore (ad esempio con aggiunta sulle info di voto, media etc etc), caratteristiche che sarebbero automaticamente ereditate dalla classe Professore, creando confusione e inconsistenze nel codice.

Un modo di procedere concettualmente in modo corretto nell'utilizzo della ereditarietà è il seguente.

Prendiamo ad esempio le classi Studente e Professore definite prima. Il test «è un» fallisce e quindi la classe Professore non è una specializzazione della classe Studente.

D'altro canto le due classi hanno dei campi in comune e non sembra un evento casuale.

In effetti, sia uno studente sia un professore sono entrambe delle persone.

La soluzione a questo problema risulta quindi naturale.

Basta *generalizzare* le due astrazioni in una classe Persona, affinché le classi Studente e Professore possano estenderla come segue.

```
public class Persona {
        public String nome;
        public String cognome;
        public String dataDiNascita;
        //…
}
public class Studente extends Persona {
        public int numeroEsamiSostenuti;
        //…
}
public class Professore extends Persona {
        public String materia;
        //…
}
```

Se fossimo partiti dalla classe Persona per poi definire le due sottoclassi avremmo parlato invece di specializzazione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

E' possibile che più classi ne estendano un'altra.

Ad esempio le classi Segretario, Direttore, Bibliotecario estendono tutte la classe Persona.

Non è possibile che una classe ne estenda più di una per volta ma solo che più sottoclassi esistano di una superclasse diretta.

Tuttavia l'ereditarietà può generare gerarchie a più livelli, ovvero una classe ProfessoreAssociato potrebbe estendere Professore che estende a sua volta estende Persona.

In tal caso Persona è, per la proprietà transitiva, una superclasse anche di ProfessoreAssociato.

Infatti un professore associato, oltre ad essere un professore è anche una persona.

La gerarchia può anche estendersi ulteriormente.

E allora potremo dire che oltre ai professori lavorano in università anche il personale amministrativo, a sua volta specializzazione di un'altra astrazione che è il dipendente dell'università rappresentato dalla classe Dipendente che insieme a studente sono messe a comune tramite la classe Persona.

A prescindere dal fatto che sia uno studente o un dipendente (sia esso professore o amministrativo) una scheda gestisce comunque informazioni su una persona.

Le schede gestite dal sistema sono quindi tutte schede di persone.

Troviamo anche una specializzazione della classe Professore rappresentato dal ProfessoreAssociato e della classe Studente rappresentato dalla classe StudenteDiDottorato.

Quest'ultimo oltre all'informazione sulla matricola e il numero di esami sostenuti presenta le caratteristiche peculiari date dal numero di articoli pubblicati su riviste scientifiche e l'informazione booleana se ha trascorso il periodo all'estero oppure no.

La gerarchia completa è mostrata in Fig. 4. Sebbene il programma possa non aver bisogno di classi che corrispondono a persone o a dipendenti, pensare in termini di tali classi può essere utile.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per esempio tutte le persone possiedono un nome e i metodi di inizializzazione, visualizzazione e modifica del nome saranno gli stessi per studenti, professori e amministrativi come mostrato nella Figura 4.

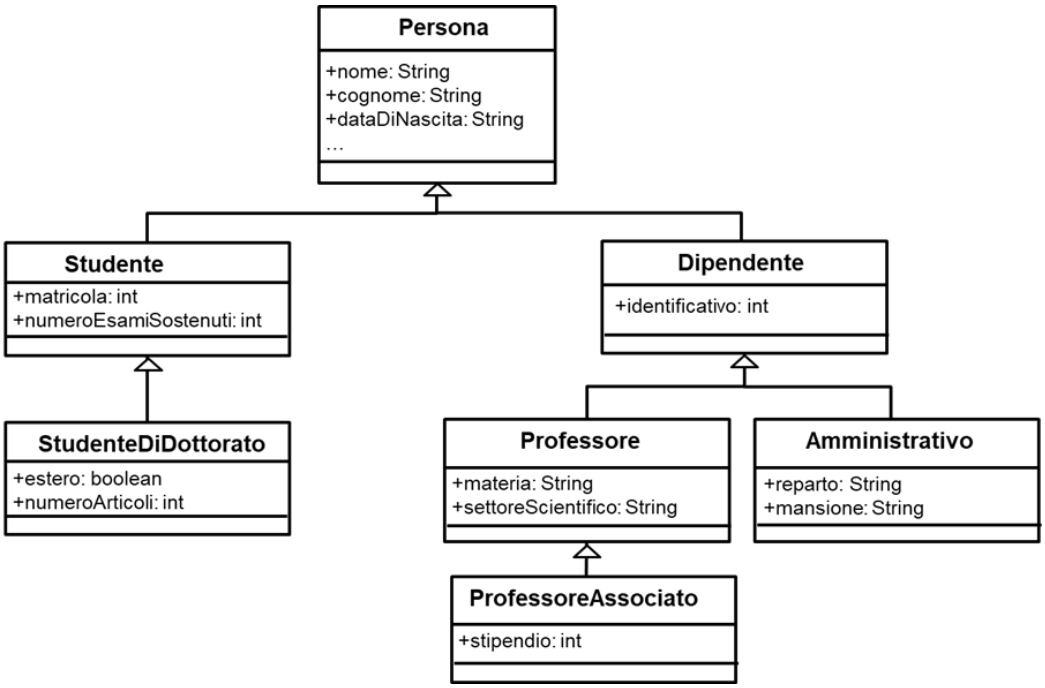


Fig. 4: Gerarchia di classi derivate da Persona

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Ereditarietà e incapsulamento

Dopo aver visto i concetti base di ereditarietà vediamo come questa si combina con il concetto di incapsulamento.

Cosa viene ereditato da una classe incapsulata?

Non vengono ereditati né i metodi privati né le variabili di istanza private, vengono ereditati solo i metodi pubblici.

Una sottoclasse non può accedere direttamente alle variabili di istanza private della sua classe base.

La sottoclasse conosce infatti il comportamento pubblico della classe base, e si presuppone che non conosca (e che non vi sia interessata) il modo in cui la classe base gestisce i propri dati che è alla base del concetto di incapsulamento che vale anche per la sottoclasse.

Tuttavia per l'accesso alle variabili di istanza private questo può essere reso possibile dalla classe derivata attraverso dei metodi pubblici (fra cui i metodi get e set realtivi ad una specifica variabile di istanza privata) che contengono riferimenti alle variabili di istanza private e che la classe derivata eredita in quanto pubblici.

I metodi privati questi sono totalmente e in ogni caso sempre inaccessibili direttamente.

Tuttavia la sottoclasse può chiamare metodi pubblici cha a loro volta chiamano metodi privati a patto che entrambi i metodi siano stati definiti nella classe base (ovvero i metodi privati possono essere usati indirettamente tramite metodi pubblici che li invoca).

Il fatto che una classe derivata non sia in grado di usare metodi privati della classe base non dovrebbe rappresentare un problema.

I metodi privati dovrebbero essere utilizzati come metodi di supporto ad altri metodi, quindi il loro utilizzo dovrebbe essere limitato alla classe nella quale sono definiti (uso interno).

Se si desidera utilizzare nei metodi delle classi derivate un metodo di supporto definito nella classe base, significa che non è un semplice metodo di supporto per la classe base, ma deve essere dichiarato pubblico affiché sia accessibili dalle sottoclassi.

Infine vengono ereditati i metodi e variabili di istanza protetti, che sono visibili anche a tutte le classi del package.

Riprendiamo l'esempio della classe Persona estesa qui di seguito con anche il costruttore e i metodi get e set.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
public class Persona {
        private String nome;
        private String cognome;
        private String dataDiNascita;

        public Persona (String nomeIniziale) { nome = nomeIniziale; }
        public setNome (String nuovoNome) { nome = nuovoNome; }
        public getNome () { return nome; }
        public void scriviOutput () {
                System.out.println («Nome: «» + nome);
        }
//altri metodi…
}
```

In questo frammento di codice della classe Persona si hanno le definizioni di tutte quelle variabili di istanza che rappresentano le proprietà comuni a tutte le sottoclassi della classe Persona ovvero le variabili di istanza nome, cognome e data di nascita.

Per motivi di incapsulamento tali variabili sono definite private e poi sono forniti anche i metodi pubblici che permettono l'accesso (controllato) a tali variabili: get e set.

Le sottoclassi la estenderanno con variabili di istanza e metodi specifici senza bisogno di ridefinire quelli pubblici dalla superclasse perché le eredita dalla classe Persona.

Consideriamo la classe Studente modificata come segue:

```
public class Studente extend Persona {
        public int matricola;
        public int numeroEsamiSostenuti;
        // altre variabili di istanza qui……
        // costruttori qui…
        }
        public getMatricola () { return nome; }
        public setMatricola (int nuovaMatricola) { matricola = nuovaMatricola; }
        public reimposta (String nuovoNome, int nuovaMatricola) {
                setNome(nuovoNome);
                matricola = nuovaMatricola;
        }
        public void scriviOutput () {
                System.out.println («Nome: «» + getNome);
                System.out.println («Matricola: «» + matricola);
        }
//altri metodi…
}
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La classe Studente come visto aggiunge la variabile di istanza matricola e numeroEsamiSostenuti alle variabili di istanza della classe Persona che in questo esempio sono definiti pubblici per semplicità ma per il principio di incapsulamento dovrebbe essere definiti privati e poi fornire i metodi get e set corrispondenti (qui fornito solo per Matricola).

Inoltre definisce i metodi pubblici reimposta e scriviOutput. Inoltre la classe Studente ha tutti i metodi pubblici della classe Persona.

Per questo motivo può invocare il metodo setNome all'interno del metodo reimposta. Lo stesso dicasi per il metodo getNome invocato dentro il metodo scriviOutput.

Di seguito un frammento di un main che usa la classe Studente:

```
public class DemoEreditarietàEIncapsulamento  {
        public static void main (String[] args) {
        Studente s = new Studente();
        s.setNome(«Mario»);
        //assegna il valore Mario alla variabile di istanza nome
        //dell'oggetto s
        // s.nome = «Giovanni» avrebbe dato errore
        s.setMatricola(1234);
        //s.matricola = 1234 permesso ma non raccomandato
        gio.scriviOutput();
        }
}
```

Si immagini di istanziare un oggetto della classe Studente come segue:

Studente s = new Studente;

Uno dei membri che un oggetto di tipo Studente eredita dalla classe Persona è la variabile di istanza nome.

Dopo aver definito un oggetto s di tipo Studente assegnamo alla variabile di istanza nome dell'oggetto s il valore «Mario».

La variabile di istanza nome della classe Studente è stata ereditata dalla classe Persona, ma si tratta di una variabile di istanza privata della classe Persona.

Questo significa che si può accedere alla variabile di istanza nome solamente attraverso i metodi definiti nella classe Persona.

Non avremmo potuto usare lo statement s.nome in quanto le varaibili di istanza che sono dichiarati privati in una classe base non sono accessibili direttamente (ovvero per messo del loro identificativo) dai metodi di nessun'altra classe, incluse le classi derivate.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Quindi s.nome avrebbe dato errore ed necessario come fatto nell'esempio usare un metodo pubblico per accedere alla variabile di istanza nome, come appunto setNome per impostare un valore come mostrato.

Per contro matricola definito come variabile di istanza specifica della classe Studente può essere impostata sempre usando il relativo metodo set (con il nome setMatricola) ma in questo caso era pure accessibile direttamente e s.matricola non avrebbe dato errore perché permesso.

Tuttavia come stile di programmazione raccomandato è preferibile usare s.setMatricola.

Infine si può stampare a video il nome e la matricola tramite il metodo scriviOutput.

Guardando meglio questo metodo era definito nella classe Persona ma anche poi ridefinito nella classe Studente.

Come detto una sottoclasse include automaticamente tutte le variabili di istanza, le variabili statiche e tutti i metodi pubblici della classe base.

I membri ottenuti dalla classe base si dicono ereditati. Le variabili di istanza, le variabili statiche e i metodi pubblici ereditati dalla classe base non sono dichiarati esplicitamente nella definizione della sottoclasse, ma diventano automaticamente suoi membri.

Esiste una solo eccezione a questa regola: in una sottoclasse derivata è possibile modificare un metodo ereditato.

Questa nuova definizione ridefinirà il comportamento del metodo solo nella classe derivata/sottoclasse. Questo è il caso del metodo scriviOutput ridefinito nella classe Studente.

Come mai se un oggetto di una classe derivata è anche un oggetto della classe base non riesce ad accedere ai suoi membri privati e in particolare alle variabili di istanza? Per preservare la correttezza del programma che potrebbe venir compromessa.

Se una variabile di istanza privata di una classe fosse accessibile dalla definizione di un metodo di una sottoclasse, ogniqualvolta si volesse accedere a una variabile di istanza privata basterebbe creare una classe derivata e accedervi da un metodo di questa classe.

Questo vorrebbe dire rendere accessibili (a chiunque voglia fare lo sforzo di definire una classe derivata) le variabili di istanza private definite in una classe.

Questo scenario illustra la problematica, ma il reale problema che si verrebbe a creare è l'involontaria introduzione di errori.

Se le variabili di istanza private di una classe fossero accessibili dalle definizioni dei metodi delle classi derivate, il programmatore potrebbe modificarne il valore inavvertitamente o in modo inappropriato (mentre con i metodi get e set si proteggono le variabili di istanza da cambiamenti inappropriati).

A questo riguardo risulta importante introdurre il modificatore di accesso protected.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Si consideri ancora la classe Studente derivata dalla classe base Persona modificata come segue:

```
public class Persona {
        protected String nome;
        ….
        //altri variabili di istanza e metodi…
}

public class DemoEreditarietàEIncapsulamento  {
        public static void main (String[] args) {
        Studente gio = new Studente;
        gio.nome = «Giovanni»
        //assegna il valore Giovanni alla variabile di istanza nome
        //dell'oggetto gio
        //gio.setNome(«Giovanni»); pure possibile

        gio.matricola = 1234;
        //gio.setMatricola(1234) pure possibile

        gio.scriviOutput();
        }
}
```

Se la variabile di istanza nome fosse specificata come protected nella classe Persona, allora la variabile di istanza nome potrebbe essere acceduta direttamente, e ancora tramite il metodo set se ancora disponibile e come la variabile di istanza matricola definita nella classe derivata (anche qui possibile accedervi tramite metodo pubblico set corrispondente).

II modificatore d'accesso protected garantisce un basso livello di protezione rispetto al modificatore d'accesso private perché qualsiasi programmatore può accedere direttamente a un membro protected definendo una classe derivata.

Per tale motivo L'utilizzo del modificatore protected è spesso sconsigliato e le variabili di istanza non dovrebbero essere specificate come protected.

Solo in rare occasioni si potrebbe voler dichiarare un metodo come protected.

Un errore in cui un programmatore neofita potrebbe incorrere è pensare che per utilizzare una variabile in una sottoclasse sia necessario dichiararla protected, invece è più che sufficiente avere a disposizione i metodi set e get nelle sottoclassi in virtù dell'incapsulamento.

D'altro canto dichiarare protetta una variabile di istanza significa renderla pubblica a tutte le classi dello stesso package.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Ereditarietà e costruttori

Dopo Il costruttore è un metodo speciale presente in ogni classe.

Infatti anche quando il programmatore non ne prevede uno esplicitamente è il compilatore che ne introduce uno al momento della compilazione, il «costruttore di default».

Serve solitamente ad inizializzare le variabili (o attributi) degli oggetti al momento dell'istanza ed ha le seguenti proprietà:

- Ha lo stesso nome della classe a cui appartiene;
- Non ha tipo di ritorno
- E' chiamato automaticamente e solamente ogni volta che viene istanziato un oggetto della classe a cui appartiene, relativamente a quell'oggetto.

Cosa succede in caso di classe derivate e quindi quando entra in gioco l'ereditarietà?

L'ereditarietà non è applicabile ai costruttori. Anche quando sono dichiarati pubblici, i costruttori non sono ereditati per un motivo semplicissimo: il loro nome.

Il nome del costruttore è il nome stesso della classe di appartenenza e si invoca attraverso una invocazione new con il nome della classe stessa.

Avendo la superclasse e sottoclasse nomi diversi non si può ereditare il costruttore.

Consideriamo il seguente esempio.

```
public class Libro {
        public String titolo, autore, editore;
        public int numeroPagine, prezzo;
        //…
}
public class LibroSuJava extends Libro
        public static final String ARGOMENTO = «Java»;
        //altre definizioni di variabili e metodi aggiuntivi
}
……
//nel main…
LibroSuJava libroSuJava = new LibroSuJava();
// viene invocato il costruttore LibroSuJava()

LibroSuJava libroSuJava = new Libro();
// viene invocato il costruttore Libro() ma non si istanzia la
classe LibroSuJava (oltre ad avere errore di compilazione)
….
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per esempio se la classe LibroSuJava ereditasse dalla classe Libro un costruttore ne erediterebbe uno che si chiama proprio Libro(). Ma un costruttore che si chiama Libro, in una classe che si chiama LibroSuJava non potrà mai essere chiamato al momento in cui si istanzia una classe LibroSuJava quando invece viene invocato il costruttore di nome LibroSuJava().

Infatti per istanziare un oggetto dalla classe LibroSuJava è strettamente necessario chiamare un costruttore di nome LibroSuJava() in modo da inizializzare tutte le variabili di istanza.

Per invocare il costruttore Libro() occorre istanziare con una new una classe di tipo Libro.

Per esempio, dato il codice:

LibroSuJava libroSuJava = new LibroSuJava();

con cui si istanzia una classe LibroSuJava e quindi si invoca il costruttore LibroSuJava(), si considera al suo posto:

LibroSuJava libroSuJava = new Libro(); istanzieremmo un oggetto della classe Libro (con la chiamata al rispettivo construttore) ma non un oggetto della classe LibroSuJava (ottenendo anche un errore in compilazione in quanto l'assegnazione non è corretta).

Quindi il fatto che i costruttori non siano ereditati dalle sottoclassi è coerente con la sintassi del linguaggio.

Tuttavia sembra violare i principi del riuso e dell'astrazione.

Ci si aspetterebbe di poter riutilizzare il costruttore della superclasse, essendo la sottoclasse anche un oggetto della superclasse dal momento che verifica la relazione «è un».

Un libro su Java essendo anche un libro deve avere tutte le caratteristiche di un libro.

In particolare abbiamo bisogno di inizializzare fra le variabili di istanza anche quelle ereditate dal Libro.

E allora perché non riutilizzare anche il costruttore della superclasse?

In realtà una proprietà del costruttore oltre ad avere lo stesso nome della classe a cui appartiene, a non avere un tipo di ritorno, a essere chiamato automaticamente (e solamente) ogni volta che è istanziato l'oggetto della classe a cui appartiene, è anche che: qualsiasi costruttore (anche di default) come prima istruzione invoca sempre un costruttore della superclasse.

Il principio di riuso e astrazione sono salvi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Consideriamo il seguente esempio:

```
public class Libro {
        // variabili di istanza titolo, autore, editore, numeroPagine, prezzo
        public Libro() {
        System.out.println(«Costruito un Libro!»)
        }
        // costruttore
        // altri metodi
}
…
public class LibroSuJava extends Libro
        public static final String ARGOMENTO = «Java»;
        public LibroSuJava() {
        System.out.println(«Costruito un Libro su Java!»)
        }
        //altre definizioni di variabili e metodi aggiuntivi
}
…
//nel main…
new LibroSuJava();
// invocato costruttore di LibroSuJava che invoca come prima istruzione il
costruttore della superclasse Libro().
```

Consideriamo ancora le classi Libro e LibroSuJava con LibroSuJava che estende Libro andando a specializzarne alcuni aspetti come visto. Andiamo ad aggiungere dei costruttori.

Immaginiamo in un main di istanziare un oggetto LibroSuJava con una new LibroSuJava(); Si consideri che **l'assegnazione di un reference non è obbligatoria per avere una istanza.**

Avendo appreso che i costruttori non sono ereditati, ci si aspetterebbe un output con la scritta «Costruito un Libro su Java!» in realtà avremo entrambe le scritte **«Costruito un Libro!» e «Costruito un Libro su Java!»**

Il costruttore LibroSuJava ha prima invocato il costruttore della superclasse Libro() e poi ha eseguito la sua istruzione.

Quindi riepilogando….

Il costruttore è un metodo speciale presente in ogni classe che serve ad inizializzare le variabili (o attributi) degli oggetti al momento si istanza un oggetto:

- Ha lo stesso nome della classe a cui appartiene;
- Non ha tipo di ritorno
- E' chiamato ogni volta che viene istanziato un oggetto
- Invoca sempre come prima istruzione un costruttore della superclasse

Quindi nel costruttore della sottoclasse è come se ci fosse implicitamente una chiamata al costruttore della superclasse Libro prima di eseguire le proprie istruzioni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

A sua volta questo vale per la superclasse qualora derivasse da un'altra classe dando luogo ad un comportamento ricorsivo.

E' cose se si usasse un riferimento implicito alla superclasse e questo venisse usato per invocare il suo costruttore.

Questo un po' ricorda quanto succede con la parola chiave this usato come reference all'oggetto corrente.

La chiamata esplicita al costruttore della superclasse viene effettuata tramite la parola chiava super.

Il reference super consente di accedere ai componenti della superclasse e in particolare al suo costruttore.

Infatti la parola chiave super è strettamente connessa al concetto di costruttore. In ogni costruttore infatti è sempre presente una chiamata al costruttore della superclasse mediante una sintassi speciale che sfrutta il reference super.

Per esempio nella classe LibroSuJava, il costruttore sarà modificato dal compilatore nel seguente modo:

```
public class Libro {
        // variabili di istanza titolo, autore, editore,
        // variabili di istanza numeroPagine, prezzo
        public Libro() {
        System.out.println(«Costruito un Libro!»)
        }
        // costruttore
        // altri metodi
}
…
public class LibroSuJava extends Libro
        public static final String ARGOMENTO = «Java»;
        public LibroSuJava() {
        super();
        System.out.println(«Costruito un Libro su Java!»)
        }
        //altre definizioni di variabili e metodi aggiuntivi
}
…
//nel main…
new LibroSuJava();
// invocato costruttore di LibroSuJava che invoca come prima
istruzione il costruttore della superclasse Libro().
```

Ecco perché il costruttore della classe Libro (senza parametri) viene invocato dal costruttore della classe LibroSuJava.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Possiamo esplicitare la chiamata a super() ma se non lo facciamo il compilatore considererà questa istruzione implicita. La chiamata ad un costruttore della superclasse è quindi inevitabile.

Il costruttore della classe Libro chiamerà il costruttore della sua superclasse Object mediante un comando super() implicito

Supponiamo di voler dotare la superclasse Libro di un costruttore che imposta un valore alla variabile titolo.

D'altronde un oggetto di tipo Libro almeno un titolo lo dovrebbe avere.

Definiamo un costruttore che prende in ingresso una stringa da impostare come titolo nella variabile di istanza titolo. Questa classe compilerà ma LibroSuJava non compilerà più.

```
public class Libro {
        // variabili di istanza titolo, autore, editore,
        // variabili di istanza numeroPagine, prezzo
        public Libro(String titolo) {
        this.titolo = titolo
        }
        //costruttore
        // altri metodi
}
```

```
LibroSuJava.java: 1:error: constructor Libro in class Libro
     cannot be applied to given types;
Public class LibroSuJava extend Libro {
       ^
     required: String
     found: no arguments
     reason: actual and formal argument lists differ in length
1 error
```

Modificando la superclasse non compila la sottoclasse.

Infatti quando viene compilata la sottoclasse il costruttore della sottoclasse ha cercato di invocare il costruttore della superclasse.

La superclasse non ha più un costruttore senza parametri come invocato dalla chiamata super() e quindi il costruttore che si cerca di invocare è inesistente.

Infatti se un costruttore viene aggiunto esplicitamente, nessun altro costruttore di default viene inserito implicitamente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Quindi l'istruzione super() inserita automaticamente nella sottoclasse LibroSuJava, prova a chiamare il costruttore della superclasse senza parametri (infatti non sono passati parametri tra le parentesi).

Per risolvere il problema, occorre modificare il costruttore di LibroSuJava in modo tale da fargli invocare il costruttore della superclasse Libro che prende come parametro di input il titolo nel seguente modo:

```
public class Libro {
        // variabili di istanza titolo, autore, editore,
        // variabili di istanza numeroPagine, prezzo
        public Libro(String titolo) {
        this.titolo = titolo
        }
        //costruttore
        // altri metodi
}

public class LibroSuJava extends Libro
        public LibroSuJava(String titolo) {
        super(titolo);
        }
        //…
}
```

Nella classe LibroSuJava abbiamo invocato esplicitamente il costruttore della superclasse che prendo in input una stringa.

Se avessimo più costruttoRi nella superclasse potremmo scegliere quale chiamare a seconda della necessità.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Il nuovo Java, Claudio De Sio Cesari, Hoepli Informatica
- Programmazione di base e avanzata con Java, Walter Savitch, Pearson

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).